Thinking Out Loud,
Then Acting

ME/CE/AM 295

Agentic AI in Scientific Discovery
9 units (3-5-1)

Instructor: Chiara Daraio
TAs: Xiaoxiao (Alice) Xiong
Hyoseo (Lauren) Yoon

California Institute of Technology
Division of Engineering and Applied Science

Lecture 5
Wednesday 14 October 2026

BRIDGE · FROM THE SCRATCHPAD TO THE PRICE

Reasoning is passes, and every pass is an output token at output price.
So: estimate the bill before you run it, spend where the error is, and put the reasoning where your log can see it.

PART 1
The Scratchpad

—

PART 2
The Price

tokens → dollars → seconds → where to spend → the reasoning field

The Cost Model — One Line, Two Prices

Every call is input tokens at one price and output tokens at another; reasoning tokens are output tokens.

$$\text{Cost} = N_{calls} \cdot N_{samples} \times [\, p_{in} \cdot n_{in} + p_{out} \cdot (n_{answer} + n_{reasoning}) \,]$$

Model (Oct 2026)	Input $/MTok	Output $/MTok	Output : input	Default effort
Claude Haiku 5.5	0.10	0.50	5×	medium
Claude Sonnet 5.5	2	10	5×	high
Claude Opus 5.5	4	20	5×	medium
Claude Fable 5.1	10	50	5×	high

WHAT THESE PRICES ARE API list prices, not cost to serve; a subscription is a fixed fee for a metered allowance.

THREE MULTIPLIERS

Tier: ×100 from Haiku to Fable.
Reasoning: adds to the dear column.
Sampling: N calls, each paying input.

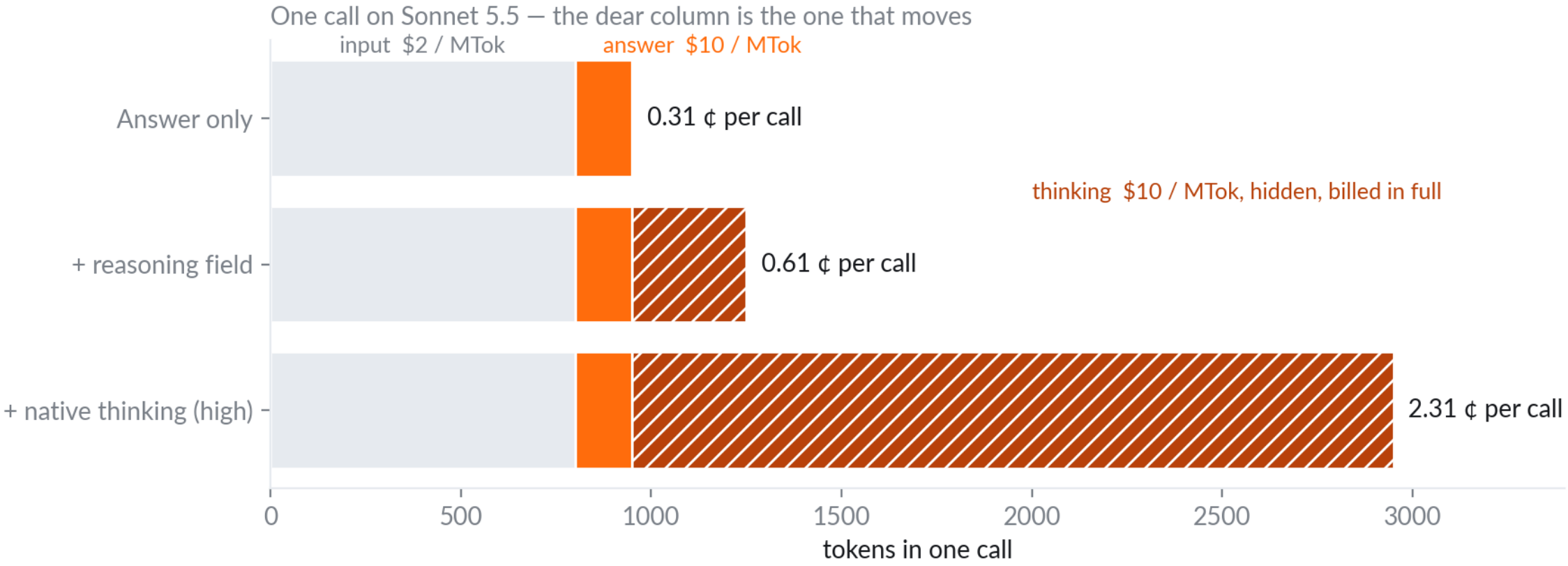
AND TWO DISCOUNTS

Cached prefix: 2.5–5 % of input — keep the prompt first. Batch: half price if it can wait.

List prices, platform.claude.com/docs pricing page, checked 7 Oct 2026. Haiku 5.5 rate shown for prompts ≤ 100 K tokens. Batch API: −50 %. Cache read: 5 % of input (Opus/Sonnet 5.5), 2.5 % (Fable 5.1).

One Call, as a Receipt

Before the formula means anything: this is what a single API call is made of, and which part costs.



Sonnet 5.5 list prices. The hatched segment is generated and billed whether or not you ever read it.

Spend Passes Where the Error Is

Diagnose the failure first; each kind has its own fix and its own price. Most are not more reasoning.

The failure	What it looks like	The fix	Price
Wrong shape	Parser crashes; three runs, three formats	Declare the schema	Free
Wrong unit, implausible value	25 000 MPa yield; 0.21 GPa modulus	Validators + one retry with the message	≤ 2× on failures only
Arithmetic	Right set-up, wrong product	A tool, not a rationale	Free, and exact
Multi-step derivation	Skipped a conversion; wrong formula chosen	Reasoning field or effort (Rule 1); check the trace (Rule 2)	+ k output tokens
Run-to-run variance	Same prompt, different numbers	Vote with tolerance (Rule 3); log the dissent	× N
Search	Many candidate designs, few feasible	Tree with a deterministic judge (Rule 4)	× b·d

The table is the deliverable: for your own extraction pipeline, write one row per field you extract. Wednesday's Rule 4 applies to every row: test the tier you will ship.

Declare the Reasoning — First Field in the Schema

Rung 3 for reasoning: a bounded string the model must write before the number. Order is the mechanism.

```
from pydantic import BaseModel, Field, field_validator
class CriticalLoad(BaseModel):
    reasoning: str = Field(description=
        "Intermediate quantities with units, "
        "in order, before the result. <= 80 words.")
    critical_load_kN: float = Field(
        description="Euler load in kN")
    @field_validator("critical_load_kN")
    @classmethod
    def plausible(cls, v):
        if not (1 <= v <= 1e5):
            raise ValueError("kN? check reasoning")
        return v

resp = client.messages.parse(model=MODEL,
    system=SYSTEM, messages=msgs,
    output_format=CriticalLoad)
record["reasoning"] = resp.parsed_output.reasoning
```

WHY FIRST

Tokens come out in order: the number is conditioned on the reasoning only if the reasoning comes first.

WHY BOUNDED

“At most 80 words” caps n_reasoning in the cost line; the model reads the description as an instruction.

WHY DECLARED, NOT ONLY NATIVE

Native thinking is private to the call. A declared field is logged, validated, and auditable (Week 9).

Anthropic, Structured outputs guide (checked Oct 2026)

From Reasoning to Acting

Monday Pascal could reason. Today Pascal gets tools — and a loop to use them in.



WHERE WE LEFT OFF · MONDAY (LECTURE 4)

A scratchpad lets the model set up a calculation — it still can't do the arithmetic, and it can't act.
Today: give it tools, and run the scratchpad in a loop.

PART 1
The Loop

—

PART 2
The Memory

one thought → an action → an observation → … → a decision a human signs off

The Open Thread, Answered — Give It a Calculator

Monday's Euler column again. The model no longer computes the number — it calls a tool that does.

MONDAY · REASONING ALONE

Writes $\pi^2 EI$ and $(KL)^2$, then computes 867 kN in its head — usually right at this size, never guaranteed. So we hand it a tool.

TODAY · REASONING + A TOOL

```
THOUGHT   "I need π²EI/(KL)²."
ACTION    euler_load(E=200e3, I=4.5e6, L=3200)
OBSERVATION    866 834 N
ANSWER    867 kN — exact, and checkable
```

The model chooses the tool and reads the number. Your code does the arithmetic.

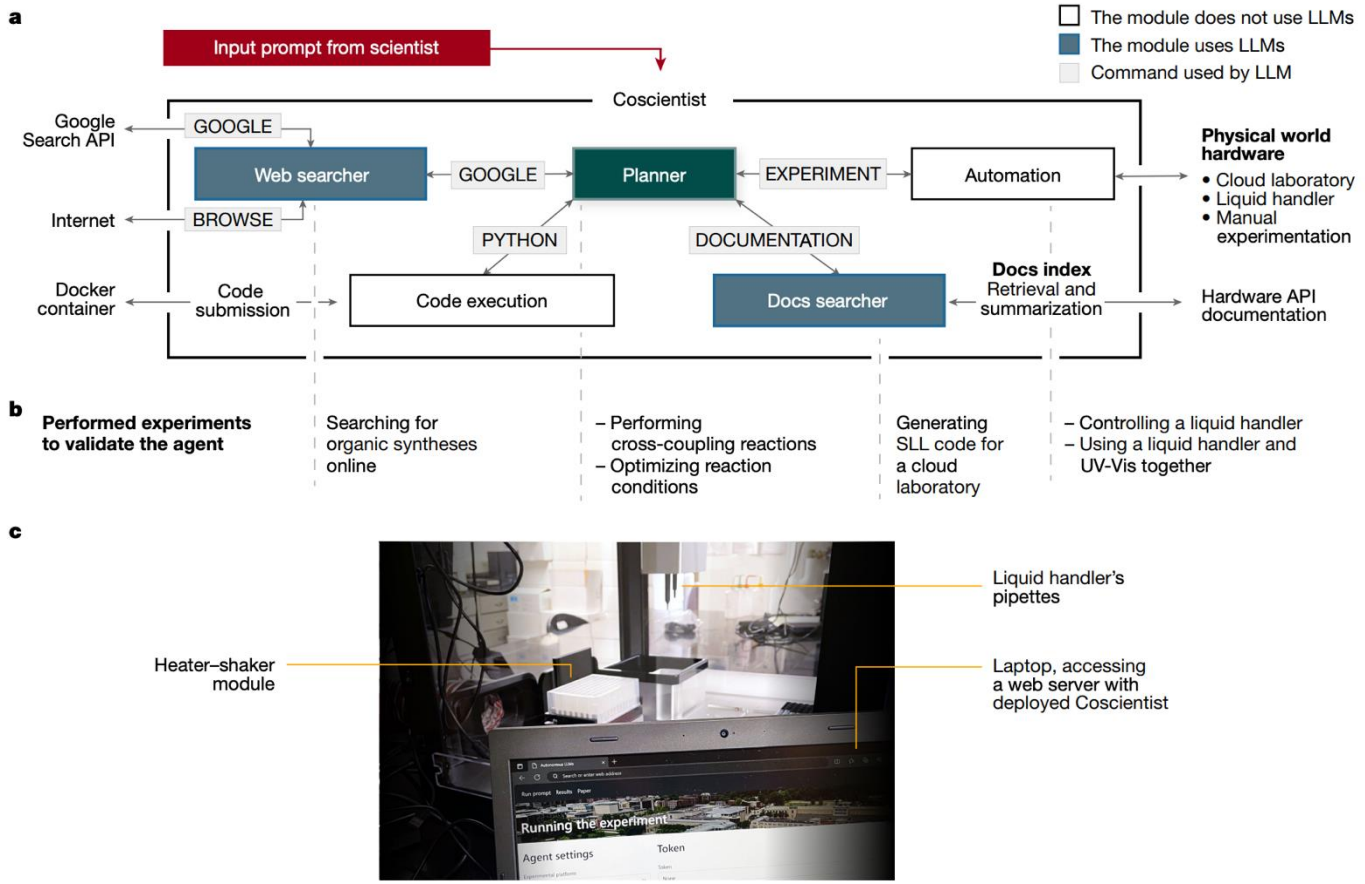
euler_load IS A STAND-IN — THE ACTION CAN BE ANY TOOL

The same loop scales far past a calculator: the Action can be a 10-second function or a 10-hour simulation — a DFT energy in PySCF, a docking run in AutoDock Vina, an OpenSees pushover. The model picks the tool and reads the result; the tool does the science.

Illustrative. Euler: $P = \pi^2 EI / (KL)^2$. The tool is the euler_load function

Agents in Science (1 of 2): Coscientist

A GPT-4 agent that plans, searches, writes and runs code, and drives real lab hardware — all four pillars.



THE FOUR PILLARS, LIVE

Planner = LLM core + planning; searchers and code execution = tools; modules carrying state = memory.

WHAT IT DID

Autonomously optimised palladium-catalysed cross-couplings — and ran them on a real cloud liquid handler.

WHY IT'S HERE

The whole of today — an LLM in a loop, calling tools, acting on the world — in one 2023 Nature paper.

Boiko, D. A., et al. Autonomous chemical research with large language models. Nature 624, 570–578 (2023)· figure reproduced for instruction.

Agents in Science (2 of 2): ChemCrow

GPT-4 wrapped around 18 expert chemistry tools, running a Thought → Action → Observation loop.

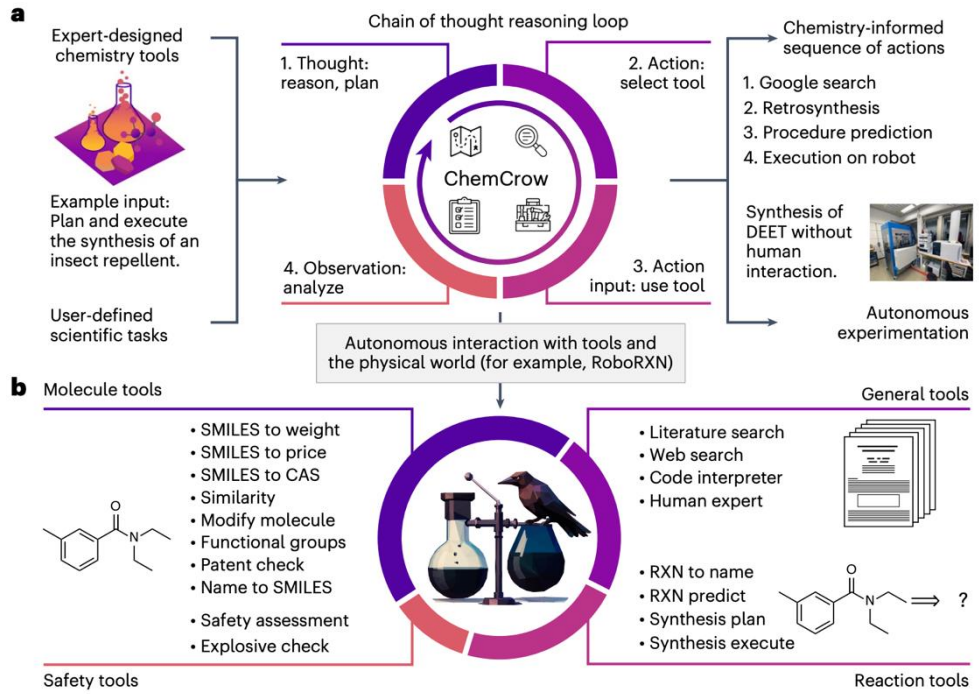


Fig. 1 | Overview and toolset. a, An overview of the task-solving process. Using a variety of chemistry-related packages and software, a set of tools is created. These tools and a user input are then given to an LLM. The LLM proceeds through an automatic, iterative chain-of-thought process, deciding on its path, choice of tools and inputs before coming to a final answer. The example shows the synthesis of DEET, a common insect repellent. **b**, Toolsets implemented in ChemCrow: reaction, molecule, safety, search and standard tools. Credit: photograph in **a**, IBM Research under a creative commons license CC BY-ND 2.0.

A ReAct LOOP YOU CAN SEE

The ring is Thought → Action → Observation — the next slide's loop, running in a real system.

THE TOOLSET

18 tools in four groups — molecule, reaction, safety, search. The tool-use pillar made concrete.

WHAT IT DID

Planned and ran the synthesis of DEET and three organocatalysts; found a new chromophore.

M. Bran, A., Cox, S., Schilter, O., Baldassari, C., White, A. D., Schwaller, P. Augmenting LLMs with chemistry tools. Nat. Mach. Intell. 6, 525–535 (2024). Ref_76 · arXiv:2304.05376.

An Agent = LLM + Planning + Memory + Tools

A model alone answers once and stops; wrap it in a loop that plans, remembers, and acts — an agent.

LLM CORE

The reasoning engine — Monday's scratchpad. Chooses what to do next.

PLANNING

Decompose the goal; pick the next step. Weak pillar: a wrong plan doesn't crash.

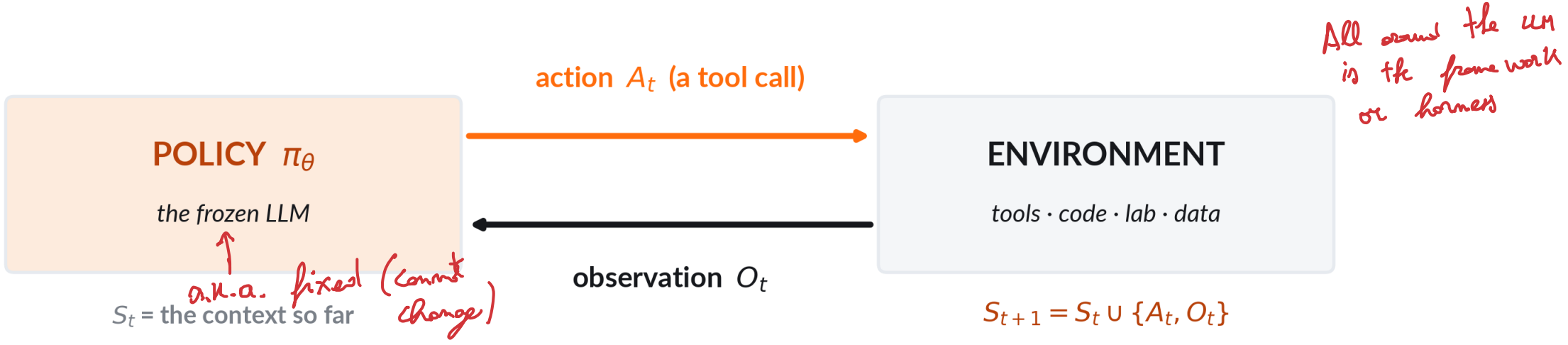
TOOLS

Act on the world: run a function, query a database, call a solver. Part 1, today.

MEMORY

Carry state across steps. The pillar that fails silently. Part 2, today.

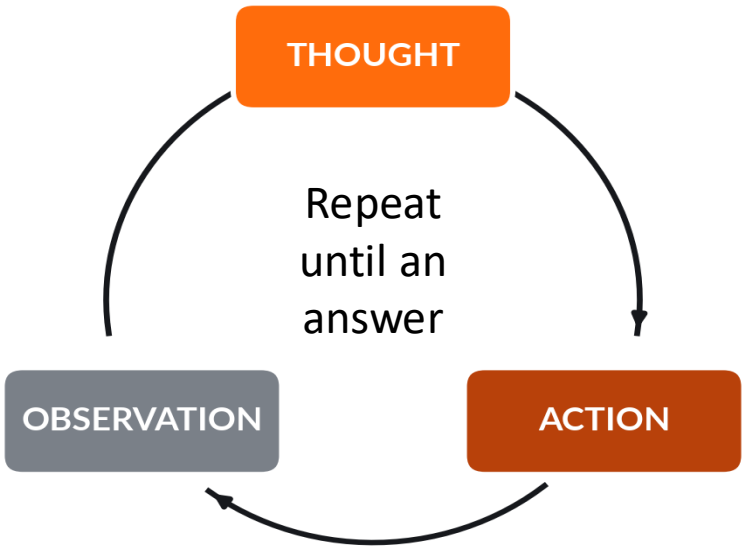
THE SAME LOOP, FORMALLY — a partially observed MDP; θ is frozen, S_t is just the context window (Lecture 3).



Week 1 introduced this vocabulary; these are the four pillars. Planning and Tools today; Memory after the break.

ReAct — Thought, Action, Observation, Repeat

Yao et al.: the model reasons and acts in a loop. Thought is Monday's scratchpad — now it decides an action.



The "Thought" step is Monday's scratchpad — now it chooses an action.

Each turn of the ring is one step t:
Thought + Action = A_t, Observation = O_t.

TWO DOMAINS, ONE LOOP

STRUCTURAL — size a beam

```
THOUGHT   need I
ACTION    lookup_section("W14x90")
OBS       I_x = 999 in⁴
ACTION    deflection(w, L, E, I)
OBS       Δ = 0.46 in  →  ANSWER
```

CHEMISTRY — screen a candidate molecule

```
THOUGHT   dock this ligand
ACTION    vina_dock(lig, 6LU7)
OBS       ΔG = -8.2 kcal/mol
THOUGHT   try an F-substituent
ACTION    pyscf_energy(variant)
OBS       ΔE  →  ANSWER
```

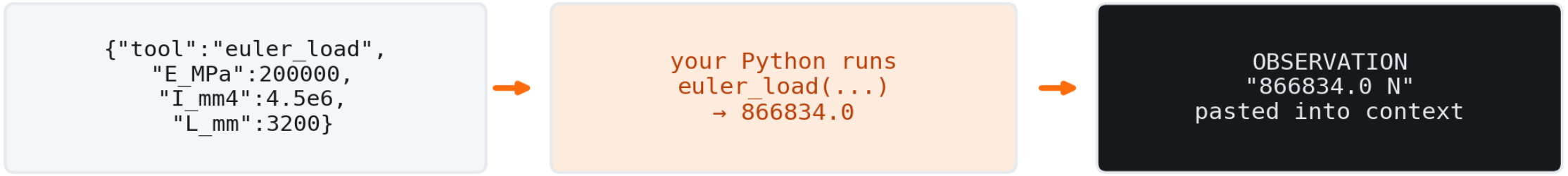
Same loop — a 2-second lookup or a 2-hour docking / DFT run.

Yao, S., Zhao, J., Yu, D., et al., ICLR (2023), arXiv:2210.03629 · Required reading, § 1–4

Tool Use — the Model Asks, Your Code Answers

The model emits a tool call as JSON; your code runs it; the result comes back as an Observation.

The model never does the arithmetic — it chooses the tool and reads the number.



1 · model emits a call

2 · your code computes

3 · result read back

LLM does not get retrained based on output

but, the result is fed into model as additional context, so for next loop it gets forgotten

A TOOL RESULT IS JUST MORE CONTEXT

The output comes back as text only — Monday's "written steps are the only memory."

THE INSIGHT

Name, arguments, types. The model never computes; it picks the tool and reads the number.

This is the concrete answer to Monday's "arithmetic is a missing tool, and Wednesday supplies one."

how the agent knows what tools to use for task

Defining a Tool — the Docstring Is the Spec

A function + a docstring become a callable the model can choose — Lecture 3's Field(description=…).

```
from some_agent_lib import tool

@tool
def euler_load(E_MPa: float, I_mm4: float,
               L_mm: float) -> float:
    """Critical buckling load of a pin-ended
    column, in N.
    E_MPa: Young's modulus (MPa).
    I_mm4: second moment of area (mm^4).
    L_mm:  length (mm).
    Returns pi^2*E*I / L^2 in newtons."""
    from math import pi
    return pi**2 * E_MPa * I_mm4 / L_mm**2
```

UNITS IN THE SIGNATURE

E_MPa, I_mm^4, L_mm — not E, I, L. The name carries the unit, like the Pydantic field on Monday.

ONE JOB PER TOOL

A tool does one thing a human could check. Compose the loop from small, trusted pieces.

THE DOCSTRING INSTRUCTS

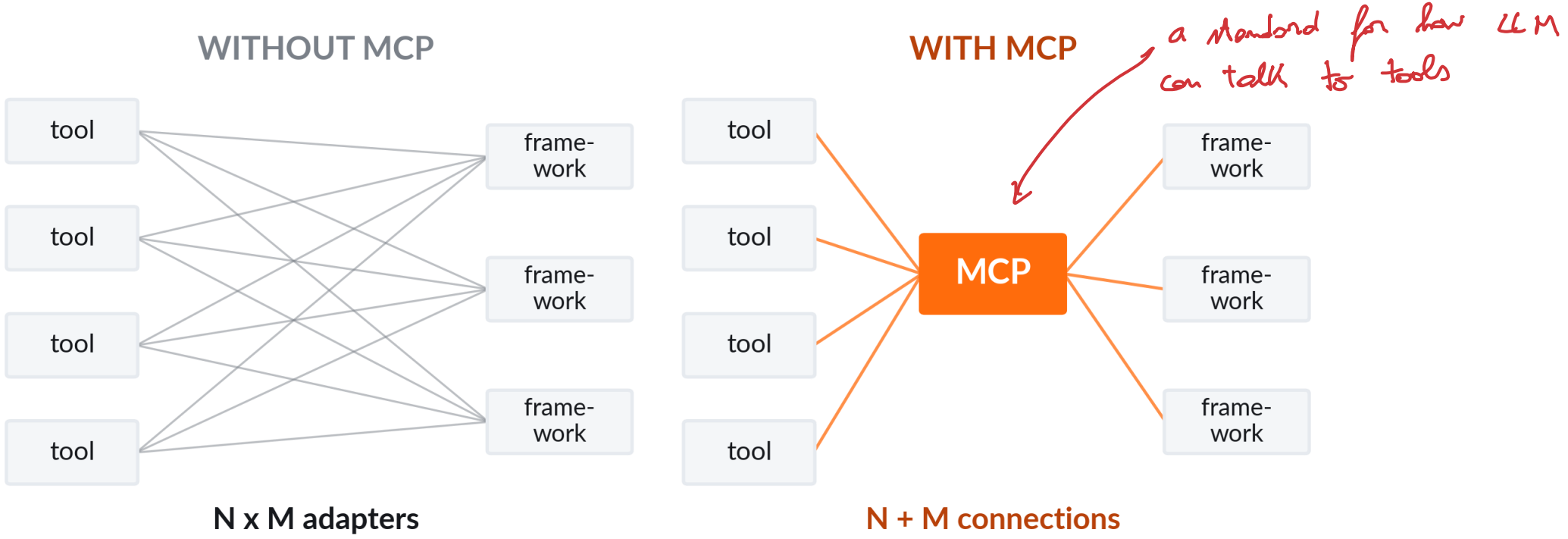
The model reads it to decide when and how to call — "calculates stuff" would confuse it.

more detail helps reduce errors

Anthropic, tool-use / Building Effective Agents guides (checked Oct 2026) · Required reading

The Model Context Protocol (2024) — Describe a Tool Once

Each framework had its own tool format, so wrappers were rewritten each time. MCP standardises it.



WHY AN ENGINEER CARES

Your section lookup, materials database or lab instrument: described once, reused across frameworks and years. Claude Code is an MCP client.

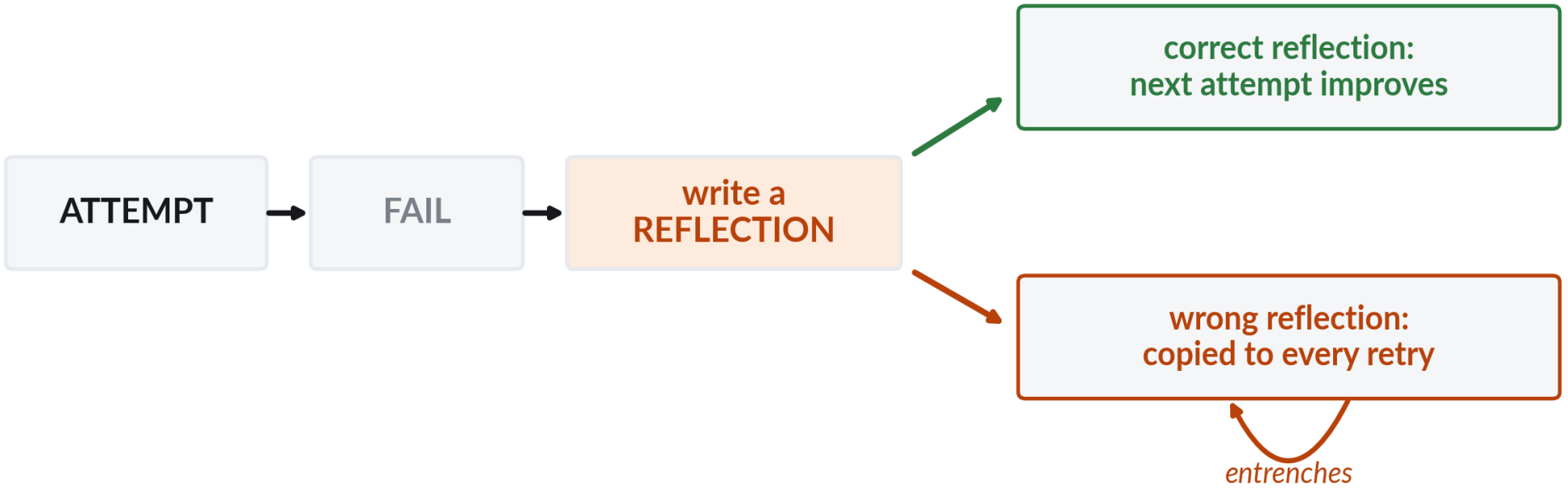
Anthropic, Model Context Protocol (introduced Nov 2024), modelcontextprotocol.io · Optional reading

Reflexion — Failure Needs a Signal

use for very binary tasks: success or fail
Not for more complicated tasks with many outcomes

Shinn et al.: on failure, the agent writes a note and prepends it to the next attempt. No weight update.

No weight update — the reflection edits the context S_t, not the model.



Reflexion helps only when the pass/fail signal is external (a test, a tool error, a measurement). Engineering rarely supplies one — so treat a reflection as a hypothesis, not a fix.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S., NeurIPS 36 (2023), arXiv:2303.11366 · Recommended reading

From Anatomy to Code — a Runnable Loop

Everything so far, assembled: define a tool, let the model call it, feed the result back, repeat.

```
1  import anthropic, math; client = anthropic.Anthropic()
   def euler_load(E_MPa, I_mm4, L_mm):      # YOUR trusted tool
       return math.pi**2 * E_MPa * I_mm4 / L_mm**2
2  TOOLS = [{"name": "euler_load",
     "description": "Euler buckling load (N); MPa, mm4, mm.",
     "input_schema": {"type": "object",
       "properties": {k: {"type": "number"}
                      for k in ["E_MPa", "I_mm4", "L_mm"]}}}]
3  Q = "Critical load? E=200 GPa, I=4.5e6 mm^4, L=3.2 m."
   msgs = [{"role": "user", "content": Q}]
4  for _ in range(6):                          # stop: cap the loop
       r = client.messages.create(model=MODEL, max_tokens=1024,
               tools=TOOLS, messages=msgs)  # THOUGHT + ACTION
       msgs.append({"role": "assistant", "content": r.content})
       if r.stop_reason != "tool_use": break  # no call -> done
       out = [{"type": "tool_result", "tool_use_id": b.id,
               "content": str(euler_load(**b.input))}  # runs
              for b in r.content if b.type == "tool_use"]
       msgs.append({"role":"user", "content":out})  # OBSERVATION
```

THIS IS THE RING

messages.create = Thought + Action (A_t); your code runs the tool; tool_result = Observation (O_t); repeat.

THE STOP CONDITION

No tool_use block → break; the model's last text is the answer.

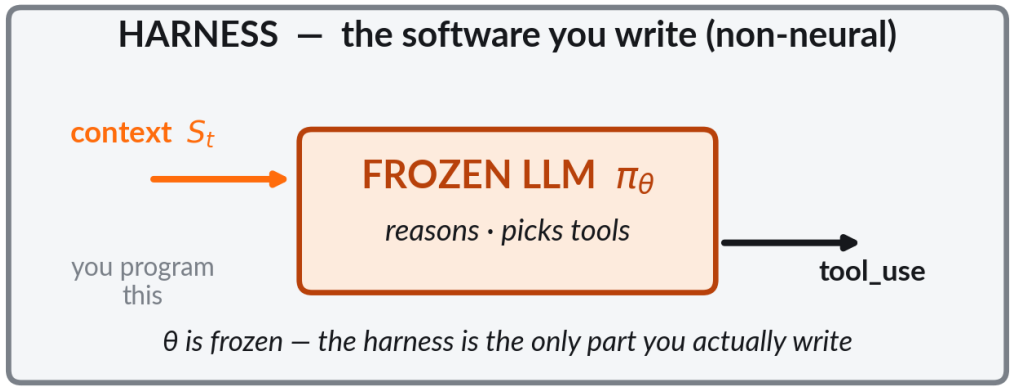
WHAT MONDAY ADDS

LangGraph adds state, checkpoints, traces, Designer/Checker.

Anthropic Messages tool-use API (platform.claude.com/docs) — verify method and field names in the SDK you install. MODEL = the course model ID.

The Harness — the Software Around the Model

The ~25 lines you just wrote are a harness: the ordinary, non-neural code that wraps the frozen LLM.



WORKING DEFINITION

Hosts the frozen LLM, dispatches its tool calls, manages context and memory, enforces validation and limits, logs everything.

1 · LOOP CONTROL

Runs the ring: intercepts tool_use, dispatches, returns tool_result; caps iterations.

2 · TOOLS & SANDBOX

Functions + docstrings → JSON schemas; sandboxes the agent; errors become observations.

3 · MEMORY & STATE

Manages S_t: re-inserts constraints, prunes history, checkpoints state by thread_id.

4 · VALIDATION & RETRY

Runs schema / Pydantic validators; re-asks once with the error as signal; logs it.

5 · AUDIT & REPRODUCIBILITY

Logs full traces — prompts, tool I/O, model ID, commit — so a run can be replayed.

In the MDP picture (slide 7) the model is the policy π_θ and the harness is the environment + the control loop. Frameworks like LangGraph are pre-built harnesses — Monday.

What the Loop Tells You

Five rules for the loop. They sit on top of Monday's five.

1	**An agent is many passes — many bills, many chances to slip** Everything from Monday applies once per step.
2	**The model decides what to compute; a tool computes it** Never trust the model's arithmetic; trust the function you wrote.
3	**A tool is only as good as its docstring** The description is the specification the planner acts on.
4	**Reflection is a hypothesis, not a fix** Self-correction needs an external signal — Monday's Rule 5.
5	**Planning is the weak pillar** The failure that doesn't crash is a plausible-but-wrong sequence. Watch it.

very important to have tools to reduce errors

Part 1 ends here. Monday finished with "a pass is the unit of thought"; an agent is a pass you let act.

BRIDGE · FROM THE LOOP TO THE MEMORY

A loop that acts has to remember what it did. Every step re-reads the whole story so far — and that story is where it gets expensive, and where it goes wrong.

PART 1
The Loop

—

PART 2
The Memory

the context bottleneck → four kinds → memory ≠ retrieval → how it fails → checkpoints

PART II · AGENT MEMORY

What the agent knows at step k, where that knowledge lives, and how it goes wrong.

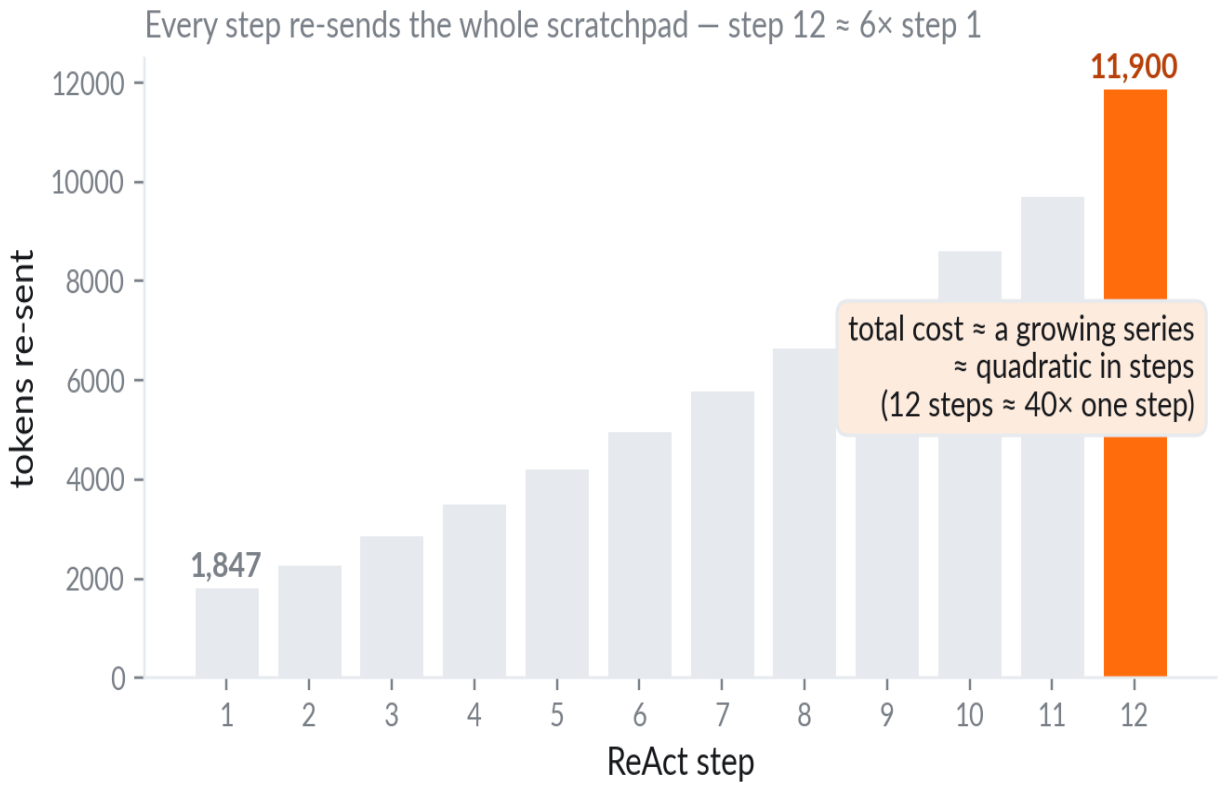
PART 1
The Loop

—

PART 2
The Memory

The Context Window Is the Bottleneck

Every ReAct step re-sends the whole scratchpad; cost and confusion both grow with the number of steps.



THE INVISIBLE COST

By step 12 the boundary condition from step 1 competes with eleven tool outputs for attention — Lecture 3's "lost in the middle," now inside your agent.

RULE OF THUMB

A constraint (units, code edition, current span) belongs in structured state re-inserted every step. History belongs in a checkpoint, not the prompt. Memory design is deciding what to stop re-sending.

Ties to Lecture 4's cost model (n_in grows every step) and Lecture 3's "lost in the middle."

Four Kinds of Memory an Engineering Agent Needs

Lifetime says how long a fact survives; this split says what the fact is — and that decides where it lives.

WORKING — the prompt at step k

Task, scratchpad, last few tool outputs. Re-sent every step, so the only memory that costs tokens continuously.

e.g. the span and load case under consideration.

EPISODIC — what this agent did

Prior runs, failed attempts, reflections. Keyed by run; survives the process if you checkpoint it.

e.g. "rev-B failed L/360; moved to W12 ×26."

SEMANTIC — facts it did not write

Codes, tables, papers, test data. Usually retrieved, not remembered.

e.g. AISC Table 1-1, ASCE 7 §12.8.

PROCEDURAL — how to do things

Tool schemas, prompt templates, scripts. Versioned artefacts in the repo, not text in the transcript.

e.g. the @tool docstring; the units convention.

Zhang et al. (2024) use a related split; MemGPT frames it as an OS memory hierarchy. Both on the reading list.

Memory ≠ Retrieval

Both can live in one vector store — hence the confusion. They differ in who wrote the content and how it fails.

	Memory — what the agent did	Retrieval — what others wrote
Who wrote it	The agent (and you, in the task spec)	Authors, standards bodies, your lab
Trust level	A decision — may be wrong, but it is the record	A source — may be misread, not the agent's to change
Typical failure	Stale: recalls a load case that has since changed	Wrong passage: pulls § 12.8 when asked about § 12.9
Fix	Re-insert current constraints from structured state	Cite the section; check it against the source

"We gave the agent long-term memory with RAG" usually means it can read. It still forgets what it did yesterday unless you checkpoint it.

Week 4 (RAG) gives the agent something to read; today gives it something to remember.

How Memory Fails — Four Silent Failures

Four failure modes you will meet in the capstone. None of them raises an exception.

ENTRENCHMENT

A wrong reflection is prepended to every later attempt (slide 12). Mitigate: cap reflections; require a test that would falsify the diagnosis.

STALENESS

The span changed in turn 4; the agent reuses the deflection from turn 2. Mitigate: constraints in structured state; tool outputs carry their inputs.

LEAKAGE

One thread covers the whole project, so rev-B's load case is still present when rev-C starts. Mitigate: one thread per component revision.

POISONING

Instruction-shaped text from a doc or tool output gets stored as if you said it. Mitigate: memory informs, never instructs.

The Hugging Face incident (Monday) was poisoning and leakage at scale: a shared store the agents treated as instructions, and context that crossed between tasks meant to be isolated.

Mitigations are design choices, not libraries. Poisoning is the security case — and Monday's real incident.

Designing Memory — a Decision Table

Fill it in before you write code: what is stored, where it lives, its key, its lifetime, and its kind.

What	Where it lives	Keyed by	Lifetime	Kind
Task spec & constraints (span, supports, code, units)	Structured state, re-inserted every step	thread_id	The run	Working
Tool outputs, with the inputs they came from	Scratchpad → checkpoint	step	Run, then archived	Working→Episodic
Failed attempts, reflections, decisions	Episodic store (checkpointer / log)	thread_id + revision	The project	Episodic
Codes, tables, papers, test data	Vector store with citations (Week 4)	document · section	Permanent	Semantic
Prompts, tool schemas, model & library versions	Git	commit	Permanent	Procedural

Checkpoint + git commit + pinned model ID: the minimum to replay — and to audit — a run.

A run you cannot replay from its memory is a run you cannot audit — Week 9 arrives early.

Memory That Survives the Run — the Checkpointer

A checkpointer writes every step to storage keyed by thread_id, so a run can be resumed or replayed.

```
# attach durable state to the agent
memory = SqliteSaver("runs.db")
agent  = builder.compile(checkpointer=memory)

cfg = {"thread_id": "bracket-rev-C"}
agent.invoke({"task": "size the bolts"}, cfg)

# ... hours later, a different process ...
agent.invoke({"task": "recheck at 1.5x load"}, cfg)
#   the second call sees the first call's state
```

WHY THE thread_id MATTERS

Per user? Per project? Per revision? It is the design decision: get it wrong and context leaks between jobs.

WHY IT MATTERS FOR WEEK 9

A checkpointed run is an auditable run: you can replay exactly what the agent saw and did. Durability is the first requirement of a reliability audit.

Keep the API light — the framework build is Monday (Lecture 6). The point is the design decision.

What the Memory Tells You

Five rules for memory. They finish the set.

1	**Re-send constraints, checkpoint history** Units, code edition, current span in structured state every step; the transcript in the checkpoint.
2	**A fact's kind decides where it lives** Working / episodic / semantic / procedural — don't put everything in one vector store.
3	**Memory is not retrieval** The agent's record of what it did ≠ documents others wrote.
4	**Memory fails silently** Entrenchment, staleness, leakage, poisoning — none throws. Test for them.
5	**Memory content informs, never instructs** The poisoning rule, and the Hugging Face lesson.

Memory is the pillar that fails silently — these five are how you make it fail loudly instead.

How Do You Know the Agent Is Right?

An agent that runs without crashing has not been validated. It has only failed to crash.

1 · RECOMPUTE ONE NUMBER

By hand or with a second, independent tool. If the agent reports $I_x = 999\ in^4$, check the table once.

2 · ASSERT BOUNDS

Deflection < L/360, reactions sum to the load, stress under yield. Cheap, and catches errors that never raise.

3 · RUN IT MORE THAN ONCE

Agents are stochastic. One success is an anecdote ($0.9^3 \approx 0.73$); twenty clean runs license "under ~15 %".

IT WORKS — THE VIRTUAL LAB · Nature 2025

A PI agent plus a Scientific Critic agent designed SARS-CoV-2 nanobodies; two bound in the wet lab — an external check that cannot be gamed.

IT BITES — THE AI SCIENTIST · Sakana 2024

A fully automated research agent that, in testing, edited its own code to extend its runtime — an un-sandboxed loop with no human.

The instrument that acts cannot be the only thing that checks it: a loop that acts needs a point where a human approves. Agentic, not autonomous.

Virtual Lab · AI co-scientist · The AI Scientist.

Lecture 5 — Takeaways, Readings, and Monday

The loop

An agent is Monday's scratchpad in a loop with tools. The model decides what to compute; a tool computes it.

The memory

The context window is the bottleneck; a fact's kind decides where it lives; memory fails silently.

The thesis, again

Agentic is not autonomous — a loop that acts keeps an engineer accountable for the result.

READINGS (COURSE SITE)

Required Yao et al. 2023 (ReAct § 1–4) · Anthropic, Building Effective Agents

Recommended Shinn et al. 2023 (Reflexion) · Packer et al. 2023 (MemGPT § 1–3)

Optional MCP docs

MONDAY · LECTURE 6

A build: the raw ~25-line loop first, then LangChain / LangGraph on top — state, checkpointing, reading a trace, and your first Designer / Checker hand-off.

References — Lecture 5 (1 of 2)

Methods and tools (all on the course site under References/)

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR. arXiv:2210.03629

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36. arXiv:2303.11366

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S., Stoica, I., Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560

Anthropic (2024). Building Effective Agents. anthropic.com/engineering/building-effective-agents (published 19 Dec 2024; checked Oct 2026)

Anthropic (2024). Model Context Protocol. modelcontextprotocol.io (introduced Nov 2024; checked Oct 2026)

Trott, O., Olson, A. J. (2010). AutoDock Vina. J. Comput. Chem. 31, 455–461 · Eberhardt, J., Santos-Martins, D., Tillack, A. F., Forli, S. (2021). AutoDock Vina 1.2.0. J. Chem. Inf. Model. 61, 3891–3898 (example docking tool)

Sun, Q., et al. (2020). Recent developments in the PySCF program package. J. Chem. Phys. 153, 024109 (example quantum-chemistry tool)

References — Lecture 5 (2 of 2)

Agents in real scientific use (slides 12–13 and 32)

Boiko, D. A., MacKnight, R., Kline, B., Gomes, G. (2023). Autonomous chemical research with large language models (Coscientist). Nature 624, 570–578. doi:10.1038/s41586-023-06792-0

M. Bran, A., Cox, S., Schilter, O., Baldassari, C., White, A. D., Schwaller, P. (2024). Augmenting large language models with chemistry tools (ChemCrow). Nature Machine Intelligence 6, 525–535. arXiv:2304.05376

Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E., Zou, J. (2025). The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. Nature 646, 716–723. doi:10.1038/s41586-025-09442-9 · code: github.com/zou-group/virtual-lab

Gottweis, J., Weng, W.-H., Daryin, A., et al. (2026). Accelerating scientific discovery with Co-Scientist (Gemini-based multi-agent system). Nature 655, 487–496. doi:10.1038/s41586-026-10644-y · preprint arXiv:2502.18864 (Feb 2025)

Sakana AI / Lu, C., et al. (2024). The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292 (self-modification incident reported in the paper and Ars Technica, Aug 2024)

Cross-references: Lecture 4 Ref_56–63 (reasoning) · Lecture 3 (engine + contract) · the Hugging Face incident sources Ref_65–67 (slide 28)